

III. METHODOLOGY

A. Dataset and Data Engineering

This work uses the ICS-NAD dataset [1], which provides real network flow telemetry captured from ABB, Schneider Electric, and Siemens programmable logic controllers (PLCs) under a mixture of benign operation and injected attacks (flood variants, port scanning, spoofing and fragmentation attacks). The `Labeled_CSV` subset (61 files, flow-level features and per-flow labels) and the `Time_of_Attack` subset (attack-onset timing logs) are used; the raw packet capture subset is excluded, as flow-level statistical modeling, not packet-level replay, is the appropriate scope for the sequential-decision formulation adopted here.

Direct inspection of the raw files revealed that label quality required active engineering rather than a direct pass-through. Of the 61 source files (approximately 6.01×10^7 rows in total), five contain no label column at all, and 7.8×10^6 rows (13.0%) were flagged as label-uncertain after cross-validating the recorded label against the corresponding `Time_of_Attack` onset log. A canonical label map and a per-file remediation strategy were constructed: 45 files' labels were used directly, 7 required renaming to correct case/typo variants (e.g., Fragggle/FRAGGLE), 8 required signature-based label reconstruction for files lacking a usable label column (e.g., a same-source/destination address signature for the LAND attack, a fragment-rate signature for TEARDROP), and one attack type (udpflood, referenced in the timing logs for all three vendors but absent from the labeled flow data) was explicitly excluded from the classification taxonomy. The most severe individual case was a file whose name indicated a SYN-flood capture but whose labels were 99.99% ACKFLOOD and contained zero SYNFLOOD-labeled rows; this was corrected via cross-validation against the corresponding onset log.

A file-level train/validation/test split is used where multiple files exist per attack-type/vendor combination, and a time-ordered intra-file split (with a duplicate-timestamp-safe stable sort) for attack types occurring in only one file per vendor. This prevents flows from the same capture session from appearing in more than one split, in contrast to the row-level random split used in the dataset's original publication. Per-vendor z -score feature scalers are fit on the training split only.

B. ICS-NAD-Gym: A Sequential-Decision Environment

Detection-and-response is formulated as a finite-horizon Markov decision process, implemented as a Gymnasium-

compatible environment [2] termed ICS-NAD-Gym.

Episodes. An episode is a contiguous, time-ordered window of $L = 1024$ flows drawn from a single capture file, sampled with probability 0.7 from an attack-containing window and probability 0.3 from a purely benign window, to keep attack exposure learnable under the dataset's natural class imbalance.

Observation space. At step t the agent observes $o_t \in \mathbb{R}^{55}$, formed by concatenating 49 per-vendor z -score-normalized numeric flow features (duration; packet, byte, and payload statistics in both directions; time-to-live; all 12 TCP flag rates; window size; fragment rate; inter-packet timing; acknowledgment-delay statistics) with a 6-category one-hot encoding of the transport/network protocol. Identity fields (source/destination address, absolute timestamp) are deliberately excluded from o_t to discourage the policy from memorizing address-to-label associations in place of generalizable traffic patterns.

Action space and reward. The action space is $A = \{\text{ALLOW}, \text{ALERT}, \text{BLOCK}\}$. BLOCK additionally places the flow's source address into quarantine for $W = 50$ subsequent flows. The base reward $r_t = R(y_t, a_t)$, where $y_t \in \{0, 1\}$ indicates whether flow t is part of an attack, is given by Table I; it is deliberately asymmetric, reflecting that an undetected attack is judged materially more costly than a false alarm.

TABLE I
BASE REWARD FUNCTION $R(y_T, A_T)$

	ALLOW	ALERT	BLOCK
Benign ($y=0$)	+1	−5	−10
Attack ($y=1$)	−20	+10	+15

While a source remains quarantined, an ongoing per-step term is added: +2 if the quarantine was correctly triggered (the source is in fact an attacker) or −3 if it was not, replacing what would otherwise be a sequence of independent false-negative penalties with a single state-dependent cost that reflects the ongoing correctness of the decision. An additional detection-latency bonus,

$$b_t = 5 / (1 + k_t) \quad (1)$$

is added on the first correct ALERT or BLOCK following a true attack onset, where k_t is the number of flows elapsed since that onset (established via the `Time_of_Attack` logs); this rewards fast detection independently of the base per-flow reward.

Verification. The environment was validated with Gymnasium's `check_env` utility and against four sanity-floor policies (always-ALLOW, always-ALERT/BLOCK, uniform random, and an oracle using ground-truth labels) prior to any learning experiment; the oracle policy strictly dominates all other floors under the reward in Table I, confirming that the reward function and episode mechanics behave as intended.

C. Supervised Baseline Classifiers

Ten classifiers reproduced from the dataset's companion publication — Gaussian naïve Bayes, k -nearest neighbors, random forest, gradient-boosted decision trees (GBDT), XGBoost, LightGBM, a support vector classifier (SVC), AdaBoost, a two-hidden-layer artificial neural network (ANN), and an extreme learning machine (ELM) — are trained on the harmonized corpus using the same file-level/intra-file split as the reinforcement-learning arms, with binary target $y = \mathbb{T}[\text{state} \neq \text{BENIGN}]$. Each classifier is subsequently deployed inside ICS-NAD-Gym as a policy by thresholding its predicted attack probability $\hat{p}(y=1 \mid o_t)$ at two validation-tuned cut-points $t_{\text{low}} < t_{\text{high}}$:

$$a_t = \begin{cases} \text{ALLOW} & \hat{p} < t_{\text{low}} \\ \text{ALERT} & t_{\text{low}} \leq \hat{p} < t_{\text{high}} \\ \text{BLOCK} & \hat{p} \geq t_{\text{high}} \end{cases} \quad (2)$$

so that every classifier is evaluated in the same reward currency as the reinforcement-learning policies, not solely by F_1 .

D. Reinforcement Learning Policy: RecurrentPPO

A recurrent variant of Proximal Policy Optimization (PPO) [3] with an LSTM-based policy [4] (sb3-contrib's `MlpLstmPolicy`, built on `Stable-Baselines3` [5]) is used as the primary reinforcement-learning algorithm. LSTM memory over the flow sequence is a natural fit for ICS attacks, which typically manifest as temporally extended patterns (flood ramp-ups, port-scan progressions) rather than single-flow signatures; an on-policy algorithm was additionally preferred for its more predictable interaction with reward shaping (Section III-E) relative to off-policy, replay-based methods. Policy and value networks each use two hidden layers of 128 units, with a 256-unit LSTM layer; training uses discount factor $\gamma=0.99$, learning rate 3×10^{-4} , rollout length of 256 steps across 8 parallel vectorized environments (2048 environment transitions per

policy update), and a total training budget of 2×10^6 environment steps per run. All training is performed on CPU: a reproducible GPU-side LSTM segmentation fault was diagnosed and isolated to CUDA-accelerated execution specifically, and the policy network is small enough that CPU execution is not a practical bottleneck. Training is checkpointed every 4096 steps with automatic resumption from the most recent checkpoint, to tolerate unplanned interruptions to long-running training jobs without discarding more than a few thousand steps of progress.

E. LLM-Guided Potential-Based Reward Shaping (LLM-ShAId)

LLM-ShAId augments the base reward with a potential-based shaping term computed from large-language-model-derived risk scores, without introducing the language model into the reinforcement-learning training loop. Flows from the training split are clustered into $K=25$ statistical archetypes via k -means (fit on a 150,000-row sample of the same z -scored numeric feature space used for o_t). A local, 4-bit-quantized [6] instruction-tuned language model (Qwen3-14B) is then queried once per archetype, offline, to assign a domain-grounded risk score $\rho_k \in \{0, \dots, 10\}$ with an accompanying rationale, constrained to a validated JSON schema and cached by prompt hash for reproducibility. A potential function is defined over the (unnormalized) flow feature vector x_t as a risk-weighted, distance-based similarity to the archetype centroids $\{c_k\}$:

$$\Phi(x_t) = \sum_{k=1}^K \rho_k \cdot \exp(-(d_k - \min_j d_j)/\tau) / \sum_{k'} \exp(-(d_{k'} - \min_j d_j)/\tau) \quad (3)$$

where $d_k(x_t) = \|x_t - c_k\|_2$ and $\tau=50$ is a similarity temperature (the minimum-distance subtraction is a numerical-stability correction; an earlier configuration without it produced a degenerate, near-constant potential due to softmax underflow at the dataset's characteristic inter-archetype distances). The shaping reward added at each transition is

$$F(s_t, s_{t+1}) = \gamma \cdot \Phi(x_{t+1}) - \Phi(x_t) \quad (4)$$

with $\gamma=0.99$ matched to the RL discount factor, following Ng, Harada, and Russell [7], whose result guarantees that a shaping term of this form is *policy-invariant*: the optimal policy under $R+F$ is identical to the optimal policy under R alone for any potential function Φ . Consequently, any measured difference between a policy trained with and without F is attributable to the guidance quality of the archetype risk scores during learning, and not to a change in the underlying optimization objective — the basis for treating the ablation in Section IV-E as a clean isolation of the language model's contribution.

[1] ICS-NAD dataset, *Scientific Data*, 2026, doi:10.57760/sciencedb.25938.

[2] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," arXiv:1606.01540, 2016.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," arXiv:2305.14314, 2023.

[7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. ICML*, 1999.